Salon Ads Package
Developer Documentation

Integration & API Guide

Contents

1 Overview **1**

2 Folder Hierarchy **1**

3 Integration Workflow **2**
- 3.1 1. Import & Cleanup . . . 2
- 3.2 2. API Replacement . . . 2
- 3.3 3. Dependencies Setup . . . 3
- 3.4 4. Addressables Configuration . . . 3
- 3.5 5. Scene Initialization . . . 3
- 3.6 6. Store-Specific Database Targeting . . . 3

4 API Reference & Usage **3**
- 4.1 AdsMediator . . . 3
- 4.2 UI Panel Ad Treatments . . . 4

5 Troubleshooting **4**

---

1 Overview

The **Salon Ads Package** is a custom, Addressables-driven, cross-promotional ad framework. It acts as a sister package to the main AdsCaller package but is capable of functioning completely standalone. It fetches ad catalogs from remote Google Sheets, caches images/videos locally, and provides dynamic UI ad placements (Banners, Interstitials, Rewarded, Large Banners, and PopAds).

2 Folder Hierarchy

Upon importing the package, your asset structure should match the following hierarchy (as provided in the reference image):

```
Assets/
  AddressableAssetsData/
  AdsCaller/
```

```
Editor/
Prefabs/
SalonAds/
    _Demo/
    Database/
        Games/
            LocalGameCatalog.asset
    Images/
        UI/
        Inter_VideoRender/
    Prefabs/
        AdBodies/
            Banner
            Interstitial
            Interstitial_landscape
            LargeBanner
            PopAd
            Rewarded
            Rewarded_landscape
        AdsCallerSalon
        CatalogLoader
    Scripts/
```

3 Integration Workflow

Follow these steps strictly when importing the package into a new or existing project.

3.1 1. Import & Cleanup

1. **Import Package:** Import the Salon Ads unitypackage into your project.
2. **Remove Online Ad SDKs (Optional):** If your project does *not* require an online ad system (like AdMob or AppLovin MAX) and will rely purely on cross-promo:
    - Delete the `Firebase` folder.
    - Delete the `StreamingAssets` folder (if specifically tied to ads).
    - Remove all third-party ad adapters and SDKs.
3. **2022Compatible Version Handling:** If you are importing the `2022Compatible` version of this package, it includes the base `AdsCaller` script and its editor. You must **delete** those scripts along with the SDK removals mentioned above.

3.2 2. API Replacement

Replace all legacy or direct ad calls in your project with calls to the central mediator. Use the `AdsMediator` singleton.

```
// Example: Showing an interstitial
AdsMediator.Instance.ShowInterstitial();

// Example: Showing a banner
AdsMediator.Instance.ShowBanner();
```

3.3 3. Dependencies Setup

The package relies on modern Unity systems for memory management and remote fetching.

1. Open the Package Manager (*Window* > *Package Manager*).
2. Install **Addressables** (Search in Unity Registry).
3. Install **Newtonsoft Json** (Add package from git URL / by name: `com.unity.nuget.newtonsoft-json`).

3.4 4. Addressables Configuration

1. Fix any script compilation errors in the console.
2. Navigate to *Window* > *Asset Management* > *Addressables* > *Groups*.
3. Create a new Addressables group (if one doesn't exist) and ensure the prefabs located in `SalonAds/Prefabs/AdBodies` are marked as Addressable with their exact keys matching the scripts (e.g., `SalonAds_Rewarded`, `SalonAds_Interstitial_landscape`).

3.5 5. Scene Initialization

In your initial/boot scene, drag and drop the following required prefabs into the hierarchy:

- `AdsCallerSalon`
- `AdsMediator` (Create an empty GameObject, attach the script manually if a prefab is missing)
- `CatalogLoader`
- `SEHandler` (Solar Engine Analytics Handler)
- `FbAnalytics` (Firebase/Facebook Analytics)

3.6 6. Store-Specific Database Targeting

By default, the package includes placeholder data. To target your specific developer account:

1. Delete the `Database` folder located inside `SalonAds`.
2. Delete the `CatalogLoader.prefab` inside `SalonAds/Prefabs`.
3. Drag in your designated, pre-configured `Database` folder and `CatalogLoader` prefab corresponding to your target platform (Google Play, iOS App Store, or Amazon Appstore).

4 API Reference & Usage

4.1 AdsMediator

`AdsMediator.cs` is your primary entry point. It implements `IOutAdsProvider` and blindly routes requests to the active provider (which will be `AdsCallerSalon` in CrossPromo mode).

```
// Standard Ads
AdsMediator.Instance.ShowInterstitial();
AdsMediator.Instance.ShowBanner();
AdsMediator.Instance.HideBanner();
AdsMediator.Instance.ShowLargeBanner();
AdsMediator.Instance.HideLargeBanner();
AdsMediator.Instance.HideAllBanners();

// Rewarded Ads (Pass success/failure callbacks)
AdsMediator.Instance.ShowRewardedAd(
    () => { Debug.Log("Reward Granted!"); },
    () => { Debug.Log("Failed or Skipped!"); }
);
```

4.2 UI Panel Ad Treatments

Attach `PanelAdTreatment_Salon.cs` to your UI panels (e.g., Settings Menu, Store Menu) to automate ad behaviors when panels open and close. You can configure it via the inspector to automatically Show/Hide Banners or Large Banners, and to show an Interstitial upon panel activation. *Note: Enumerations for behaviors are stored in `EnumsJar_Salon.cs`.*

5 Troubleshooting

- **Ad UI not showing / Null Reference Errors:** This usually happens if the Addressables group is not set up correctly. Ensure the keys exactly match what is requested in `AdsCallerSalon.cs` (e.g., `SalonAds_Banner`).
- **Blank Ads / Ad Requests Failing Silently:** This indicates the runtime catalog does not have the required data for that specific ad type. Verify that `CatalogLoader` successfully fetched the remote Google Sheets data and that the active package name matches the entries.
- **Ghost AdsMediator Warning:** If you test in the editor without placing `AdsMediator` in the scene, a "Ghost" instance is spawned automatically to prevent null reference exceptions. This ghost only exists in the editor and will **not** be included in live builds. Always place the mediator in the boot scene.